Name- Lakshita Kabra

Roll No: 2401017

Assignment: 4

Course Name: Database Management System

Submitted To:- Miss Mansi Kajal

Section-A

1. what is a transaction in DBMS?

→ A transaction in DBMS is a sequence of one or more database operations (like INSERT, UPDATE, DELETE) that are executed as a single unit of work.

In simple terms:-

→ A transaction is all-or-nothing:

Either all operations succeed → changes are saved

or any fail occurs → everything is undone.

2. Explain ACID properties.

→ ACID properties define the rules that ensure reliable and safe database transactions in a DBMS.

- A = Atomicity
  (All or nothing)
  → A transaction is treated as a single unit
  - if all operations succeed → commit
  - if any operation fails → rollback
- C = Consistency
  Database must always remain in a correct and valid state.
- I = Isolation
  Multiple transactions run independently without affecting each other.
- D = Durability
  Once a transaction is committed, it is permanently saved.

3. what is Concurrency Control?

→ Concurrency control in DBMS is a technique used to manage multiple transactions executing at the same time so that the database remains correct and consistent. It ensures that simultaneous transactions do not interfere with each other.

4. what is a deadlock?

→ A deadlock in DBMS is a situation where two or more transactions are stuck forever, each waiting for the other to release a resource.

5. what is serializability?

→ Serializability in DBMS is a concept used in concurrency control to ensure that the result of executing multiple transactions together is the same as if they were executed one by one.

6. what is recovery in DBMS?

→ Recovery in DBMS is the process of restoring the database to a correct and consistent state after a failure (like system crash, power failure, or software error).

7. what is a checkpoint?

→ A checkpoint in DBMS is a mechanism used in recovery to save the current stable state of the database so that, in case of a failure, the system doesn't have to restart recovery from the very beginning.

Section-B

→ Explain ACID properties using banking example.

- Let's take a scenario

Transfer ₹1000 from Account A → Account B

Steps:

1. Deduct ₹1000 from A
2. Add ₹1000 to B

This whole process is one transaction

ACID properties

1. Atomicity

Either both steps happen or none

if ₹1000 deducted from A but not added to B. then System will rollback.

2. Consistency

Database must remain correct before and after

Before transfer → total money = ₹10,000

After transfer → still ₹10,000

No money lost or created.

3. Isolation

Transactions should not affect each other

while transfer is happening, another user checking balance. It should not see half-completed data.

4. Durability

Once transaction is completed → it is permanent data remains same if system crashes.

→ Illustrate a deadlock situation with two transactions.

→ Scenario

Two transactions (T_1 and T_2) and two Resources (R_1 and R_2)

1. T_1 locks R_1
2. T_2 locks R_2
3. T_1 Requests R_2 (but R_2 is held by T_2) → T_1 waits
4. T_2 Requests R_1 (but R_1 is held by T_1) → T_2 waits

Deadlock Situation

- T_1 is waiting for T_2
- T_2 is waiting for T_1

Neither can proceed → System Stuck.

→ Explain Serial vs Concurrent execution with example.

- Serial Execution

Transactions are executed one after another.

ex: Two Transactions:

T_1: transfer ₹100

T_2: Checks balance

Execution Order:

T_1 → Completed

T_2 → Starts after T_1 finishes

No overlap between transactions, simple and always correct.

- Concurrent Execution

Transactions are executed simultaneously (overlapping in time).

Same transactions:

T_1: Transfer ₹100

T_2: check balance

→ A System crashes during transaction. Explain

Section-C

→ A banking system transfers money between accounts. Explain how ACID properties apply.

- Transfer ₹1000 from Account A → Account B

Steps:

1. Deduct ₹1000 from A
2. Add ₹1000 to B

This is one transaction

- Atomicity

if ₹1000 is deducted from A but not added to B system will rollback and restore A's balance.

- Consistency

Before transfer → Total money = ₹19,000

After transfer → still ₹19,000

No money is lost or created.

- Isolation

Other transactions should not see incomplete data.

- Durability

Once transaction is done - it is permanent.

→ Two transactions update same data simultaneously. Identify possible issues.

- Lost update: One transaction update is overwritten by another.
- Dirty Read :- A transaction reads uncommitted data
- Inconsistent Data → Due to overlapping updates data becomes unreliable.

→ A System crashes during transaction. Explain recovery Steps.

- when a system crashes during a transaction, the database must recover to a consistent state so that no incorrect or partial data remains. This is handled using recovery mechanisms in DBMS.

Recovery Steps are:-

1. Detect the crash
   when the system restarts after failure, the DBMS detects that a crash occured (power failure, system error etc)
2. User log file
   The DBMS maintains a log file that records all operations:
   - Start T_1 → transaction started
   - write T_1 → data modified
   - COMMIT T_1 → Transaction Completed
   - ABORT T_1 → transaction Cancelled
3. Identify Transaction States
   After crash, transactions are divided into two types
   - Committed → These were Completed before crash
   - Uncommitted → These were incomplete at crash time.
4. Perform Recovery operations
   Undo (Rollback)
   - Applied to Uncommitted transactions
   - Reverses all their changes
   - Ensures no partial data remains

Section-D

→ Draw a transaction timeline and identify conflict.

- We have two transactions:

  T_1: Read and write data item X
  T_2: Read and write data item X

Transaction Timeline (Schedule)

Time
T_1 : R(X) W(X)
T_2 : R(X) W(X)

order: R1(X) → R2(X) → W1(X) → W2(X)

A Conflict occurs when:
→ Two operations belong to different transactions
→ They access the same data item
→ At least one operation is a WRITE

In $R_1(X)$ and $R_2(X)$
Both are READ → No Conflict
In $R_2(X)$ and W1(X)
Different transactions
and Same data X so → Conflict
In W1(X) and W2(X)
Different transaction but Both WRITE → Conflict

→ Explain one Recovery Technique

- Log-Based Recovery Technique

This is most commonly used recovery method in databases.

A log file is maintained by DBMS that records every operation performed by transactions. It is stored on stable storage so it is safe even if the system crashes.

For each Transaction (T), the log stores:

- START → Transaction begins
- READ/WRITE → Data operations
- OLD VALUE → value before change
- NEW VALUE → value after change
- COMMIT T → Transaction successful
- ABORT T → Transaction Cancelled

How it works→

After a crash, the system uses log to recover.

1. UNDO (Rollback) → Applied to uncommitted Transactions.
2. REDO (Roll Forward) → Applied to Committed Transactions